

Efficiently Linking Unstructured Data for Multi-step Reasoning

Anonymous Author(s)

Abstract

Modern LLMs and AI agents increasingly support data engineering workflows that integrate evidence from unstructured sources. Such pipelines typically do data retrieval, integration, and ranking before proceeding to more complex agentic reasoning or actions, e.g., for scientific discovery. The **core retrieval problem** in these workflows jointly executes multi-attribute filtering, multi-vector search, exact relational joins, and thresholded embedding-similarity joins. Given a planned query and monotone scoring function, our DASE query engine constructs and ranks candidate evidence tuples. It comprises (i) a multi-step reasoning query model over structured predicates, multiple vectors, and relational links; (ii) SEMJI, a sparse materialized embedding-similarity join index for rare near-neighbor pairs; and (iii) a co-designed execution layer that combines predicate-aware ANN traversal, batched access, and threshold-based score aggregation.

On scientific-discovery workloads, DASE retrieves candidate evidence for multi-step reasoning queries $6\times$ to $46\times$ faster than strong RDBMS, rerank, and vector-database baselines at comparable recall; and for tasks that require semantic-operator post-processing, DASE acts as a high-recall prefilter that makes downstream LLM evaluation both cheaper and more accurate—e.g., on SemBench E-Commerce it improves BigQuery quality from 0.67 to 0.80 while cutting cost from \$2.42 to \$0.54.

ACM Reference Format:

Anonymous Author(s). 2027. Efficiently Linking Unstructured Data for Multi-step Reasoning. In *Proceedings of SIGMOD Conference on Management of Data (SIGMOD '27)*. ACM, New York, NY, USA, 14 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Novel database and AI systems have emerged that construct answers by integrating information from multiple unstructured sources, leveraging LLMs to reason about semantic content and relationships. Examples of such *agentic data engineering systems* include “deep research” [18, 50] and “wide search” agents [6, 19, 22, 46, 47], “co-scientists” [15, 17], LLM-based information extraction tools [1, 37], and unstructured-to-structured query systems based on *semantic operators* [26, 30, 35, 39]. Despite fundamental differences in architecture and scope, these systems generally share a core operation: given a user query, they decompose it into plans, where an initial *multi-step reasoning* retrieval primitive finds and assembles entity

combinations that best satisfy multiple search criteria. These integrated results are then fed into pipelines containing rich agentic reasoning steps, which may perform aggregate reasoning or initiate other actions. Our paper targets the initial data-retrieval task and layer: given a planned query and scoring function, we construct and rank candidate evidence tuples. Our LLM-based planner, which decomposes natural language tasks into retrieval and agentic steps using a combination of chain-of-thought reasoning [44], tool-calling [49], and ReAct-based reasoning [50], is beyond the scope of the paper.

Our team, in collaboration with chemists and life scientists, is building an agentic data engineering tool to accelerate scientific discovery. Two initial classes of applications involve discovering candidate molecules with pharmaceutical benefits and engineering RNA sequences with therapeutic properties. Both require reasoning over mostly natural-language data that has been preprocessed by (1) indexing document paragraphs with embeddings, as in retrieval-augmented generation (RAG) [14], and (2) extracting knowledge-graph-like annotations of those paragraphs [32, 36] and indexing annotation labels and values by their embeddings. We use molecular discovery as the running example; the RNA workload has a similar retrieval structure. A molecule’s structure can be encoded as a string (SMILES) [45], a bit vector (ECFP) [34], or as a specialized embedding (ChemBERTa) [8] depending on the downstream reasoning steps required. Accompanying textual passages, as well as knowledge graph properties such as outcomes or experimental settings, might also be embedded using standard text or multimodal embeddings.

Example 1.1 (Agentic data engineering for molecular discovery.).

The process of discovering and optimizing molecules or RNA starts with an information gathering stage, i.e., a query that assembles candidate results. Since evidence is often fragmented across documents and their extracted annotations, a query about a molecule might (1) approximately match against a ChemBERTa embedding, extracted from the text into a knowledge graph, but linked back to the paragraph in which it appears; (2) match on an extracted organism; (3) match against embeddings of experimental conclusions. All of these matches must be joined to ensure they are within a common paper; and ranked using a function that **combines all of the individual matches**. Finally, a ranked list of promising candidate molecules from this query stage is often fed into additional tools, e.g., to optimize their chemistry, or to generate new molecules using fine-tuned LLMs.

At the data layer, this process is a ranked select-join query over a graph representation of documents, paragraphs, and annotations. The graph contains explicit structural links and implicit links induced by embedding distance. The query combines filtering over structured attributes, semantic similarity search over paragraph or annotation embeddings, exact relational joins, and thresholded embedding-similarity joins. Consider a real-world example drawn from our application:

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SIGMOD '27, Huntington Beach, CA USA

© 2027 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM

<https://doi.org/XXXXXXX.XXXXXXX>

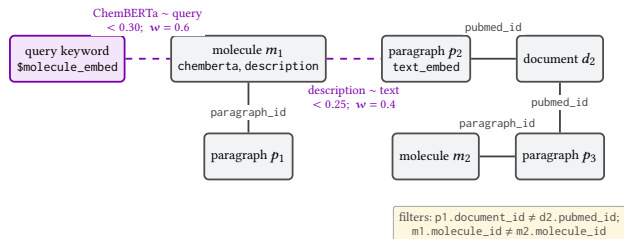


Figure 1: A graph pattern for the query described in Example 1.2, corresponding to agentic planner-generated SQL (Listing 1). The leftmost keyword-input leaf, `$molecule_embed`, matches molecule variable m_1 ; dashed purple edges are thresholded embedding-similarity predicates whose distances form the ranking score, and solid edges are exact joins.

Example 1.2. Building upon Example 1.1, a scientific-discovery query might be based on the observation that small molecules may have similar characteristics even if they are not particularly similar in ChemBERTa embedding space. Thus we might seek to *find related molecules*, by matching on a first document, then **linking and joining via text description embedding** with other documents, then traversing (joining) to their extracted molecular descriptions. Figure 1 visualizes this. We will formalize and revisit this query in subsequent examples.

The above workload requires joins to link together annotations, paragraphs, and papers. Conventional vector DBMSs such as Pinecone or Weaviate [33, 43] do not natively plan multi-table joins and joint multi-signal ranking, so an external query processor must stitch such operations together. Milvus [42], designed for hybrid search, supports logical predicates and vector retrieval but does not provide relational join planning. We therefore study how to implement multi-step reasoning queries in a relational DBMS with approximate-nearest-neighbor (ANN) vector-index support.

As we demonstrate in Section 5, standard RDBMS execution strategies for combining predicates with approximate-nearest-neighbor search [16, 27] perform poorly on join-heavy multi-step reasoning queries and other semantic-query tasks [23]. Similarly, information-retrieval-style reranking [5, 29] can incur low recall. Multi-step reasoning queries therefore introduce challenges in query indexing, optimization, and execution, which we address with the following contributions:

- **Query formalism for agentic systems.** We formalize ranked multi-step reasoning queries that support structured filters, vector scores, and cross-table joins.
- **Indexing strategy.** We develop SEMJI, a materialized embedding-similarity join index that represents near-neighbor pairs as a relational access path for top- k multi-step reasoning queries.
- **Algorithm/system co-design.** We extend PostgreSQL’s ANN-index support with batched access and develop DASE, which combines predicate-aware ANN traversal, filtered score streams, and threshold-based aggregation to jointly optimize attribute predicates and vector scoring across relational joins.
- **Evaluation.** We evaluate throughput and recall on semantic-query benchmarks and a scientific-discovery workload, including an optional transformed-space SEMJI construction for anisotropic

or shared-context endpoint embeddings that make a raw distribution-derived materialization radius unreliable.

To ensure reproducibility, our code, data, and full technical report are available at <https://anonymous.4open.science/r/DASE-2C61/>.

2 Query Model and Index Strategy

This paper targets query answering tasks over unstructured data that has been indexed and annotated within a local relational DBMS, which may include both approximate nearest-neighbor search over embeddings, as well as logical predicates over relationships.

Data model. Our underlying data model is based on object-relational tables, representing raw document and file content. Tables are linked through foreign key relationships. Using a combination of user-defined functions and semantic operators [30, 37], indexed data may be parsed, mapped to an embedding, and then analyzed using LLM operators such as *semantic map* to extract structured information from the parsed content; *semantic filter* to test tuples’ content against a prompt, etc. The results of semantic maps may be saved as labeled *annotations* in a structured relational table, in a controlled vocabulary, with explicit provenance links to the source content, and with embeddings over their values.

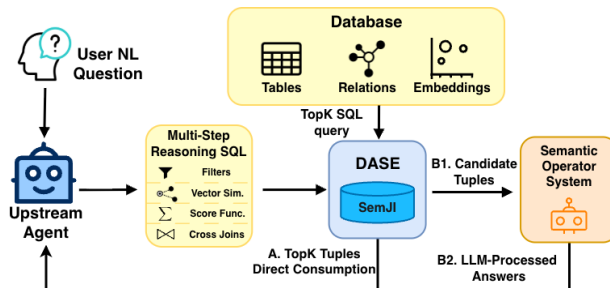


Figure 2: DASE interactions with other system components.

2.1 Top- k Multi-Step Reasoning Queries

Our target use case involves an upper-layer agentic system, performing deep or wide search, or another form of multi-step reasoning over a user’s natural-language question. This creates an agentic pipeline, in which the agent, or a planning frontend, creates an initial retrieval and integration task involving a structured query over a local dataset. This *multi-step reasoning query* combines multi-attribute filtering, multi-vector similarity scoring, and cross-table joins, together with a scoring function that aggregates these signals into a unified ranking objective. Our DASE engine executes this query and returns the top- k results. We assume the scoring function is supplied by the upstream agent. DASE is agnostic to its origin; in our own research agent, the weights are trained using logistic regression over a *calibration set* [30] of labeled samples. These assumptions are consistent with Fagin’s monotonicity assumption [12] and thus admit efficient threshold-based top- k evaluation. DASE then outputs a *ranked list*, which the upstream agent consumes in one of two modes:

1. *Direct consumption.* When the agent’s question reduces to finding the best-matching combinations of evidence—a typical pattern in wide search and multi-objective discovery—the ranked output is

the answer. Cross-encoders [29] are a common way for the agent to map per-source signals into a unified ranking.

2. Prefilter for semantic operators. For tasks that require a final LLM judgment—for example, evidence validation, entity resolution, or answer synthesis—DASE returns a high-recall candidate set to a downstream semantic-operator system [30, 35, 39]. This realizes the standard calibrated prefilter–LLM pattern: cheap proxy signals prune candidates before expensive semantic evaluation [30]. In our setting, the prefilter is a ranked, thresholded select–join query over embedding distances and structured attributes.

In either mode, DASE executes the agent-supplied top- k scoring function over structured attributes, multiple embedding spaces, and cross-table joins. We now formalize this query.

2.1.1 Query Model. A multi-step reasoning query Q is formally defined by the triple $(\mathcal{P}, \mathcal{S}, \mathcal{J})$, which characterizes the feasibility space, ranking objectives, and relational dependencies.

Predicates ($\mathcal{P}$): $\mathcal{P} = \{p_1, \dots, p_{|\mathcal{P}|}\}$ is a set of heterogeneous predicates over 1 or more **structured components** (which may be the results of a join or Cartesian product). This set may be empty ($\mathcal{P} = \emptyset$), indicating an unconstrained search space. Regardless of their specific semantic types (such as range constraints, set memberships, or probabilistic filters) each $p_i \in \mathcal{P}$ can be conceptually modeled as a Boolean indicator function $p_i : \mathcal{R} \rightarrow \{0, 1\}$. For a given query subexpression q and any resulting row instance $r \in q(\cdot)$, $p_i(r)$ logically evaluates to 1 if r satisfies the constraint, and 0 otherwise.

Similarity Join Conditions ($\mathcal{J}$): $\mathcal{J} = \{(e_{1,i}, e_{2,i}, dis_i, \tau_i)\}_{i=1}^{|\mathcal{J}|}$ is an optional set of relational constraints ($\mathcal{J} = \emptyset$ denotes a standard single-table query). Each condition specifies a thresholded embedding-similarity join between **unstructured components** across tables: a link is established when the embedding distance satisfies $dis_i(e_{1,i}, e_{2,i}) \leq \tau_i$. Thus, distances are lower-is-better feasibility predicates, not ranking scores.

Scoring Model ($\mathcal{S}$): $\mathcal{S} = \text{score}_f(s_1, \dots, s_{|\mathcal{S}|})$ defines the unified ranking objective. To guarantee a definable top- k retrieval, the query must specify at least one scoring signal ($|\mathcal{S}| \geq 1$). Every s_i is a *higher-is-better affinity*: an embedding distance d is converted by a calibrated non-increasing transform ϕ_i , e.g., $s_i = \phi_i(d) = -d$; and a scalar attribute is mapped to a desirability score by an analogous transform. Components therefore include (1) query-to-object affinity $\phi_i(dis(\mathbf{q}_v, e_i))$, (2) relational-link affinity $\phi_i(dis(e_{1,i}, e_{2,i}))$, and (3) structural-attribute desirability. The aggregation function f is coordinatewise non-decreasing (for example, a nonnegative weighted sum), so higher component affinities cannot lower the final score. The goal is to retrieve the top- k tuples that satisfy $\mathcal{P}$ and $\mathcal{J}$ while maximizing $\mathcal{S}$.

As discussed above, the scoring signals and weights are provided by an upper-layer agentic system (e.g., through calibration or learned rankings [7, 25, 48]) or by direct user specification; our methods are agnostic to their origin, provided the resulting component scores obey the higher-is-better, monotone contract above.

Our task extends classic rank aggregation [12, 40]. Traditional settings find top answers by aggregating *independent* scores that evaluate the query against individual records. In contrast, our core challenge is computing dynamic, *interdependent* scores to evaluate embedding-similarity join conditions on the fly. Reconciling these

```

1 SELECT m1.molecule_id, m2.molecule_id, d2.pubmed_id,
2       semantic_score(m1, p2, $molecule_embed) AS
3       semantic_score
4 FROM molecule m1
5 JOIN paragraph p1 ON m1.paragraph_id = p1.
6   paragraph_id
7 JOIN paragraph p2 ON p2.text_embed <=> m1.
8   description_embed < 0.25
9 JOIN document d2 ON p2.pubmed_id = d2.pubmed_id
10 JOIN paragraph p3 ON d2.pubmed_id = p3.pubmed_id
11 JOIN molecule m2 ON p3.paragraph_id = m2.paragraph_id
12 WHERE m1.chemberta_embed <=> $molecule_embed < 0.3
13 AND p1.document_id <> d2.pubmed_id
14 AND m1.molecule_id <> m2.molecule_id
15 ORDER BY semantic_score DESC LIMIT 100;
```

Listing 1: Abbreviated molecular-relationship query pattern for $\$molecule_embed$, for Example 1.2.

highly coupled data-to-data affinities with structured logical predicates, while avoiding exhaustive search and leveraging indexing, is the primary computational hurdle in retrieving the top- k results.

Example 2.1. Listing 1 gives an abbreviated SQL query for Example 1.2 (eliding result-formatting details). The `semantic_score` function computes a higher-is-better result, by negating the calibrated weighted combination of the lookup and join distances.

Standard ANN indices fall short here for two reasons. First, as they are implemented in engines such as PostgreSQL, they are designed for 1-to- N point lookups, making them **unable to execute M-to- N embedding-similarity joins following a block nested loops pattern**. Instead, they essentially only support tuple nested loops joins. Second, because they **cannot jointly evaluate the multi-signal scoring function**, the true top results for the overall query might be buried arbitrarily deep within the ANN’s ranked stream, precluding early termination.

The M-to- N embedding-similarity pattern is a critical part of many object-linking tasks, such as record linking [9] or textual similarity detection. In many cases, we will have a block of M items from the left relation (collection) that are highly similar to N items in the right relation; the final answer set may be ranked with a holistic combination of score components (the multi-signal scoring function) including the similarities of each similarity step, as well as the intrinsic scores of the original items versus specific query constraints. We thus develop an alternative approach: **precompute and materialize** partial answers based on distributional knowledge, then expand the answer frontier as needed. This decouples join discovery from scoring and supports threshold algorithms [12] for earlier termination.

2.2 Precomputing Semantic Join Tail-Events

Evaluating an embedding-similarity join naively requires distances for a Cartesian product of endpoint embeddings. In high-dimensional, approximately isotropic embedding spaces¹, pairwise distances concentrate in a dense bulk, while unusually close pairs occupy a small left tail [41]. The join candidates that carry semantic affinity are

¹“Approximately isotropic” means that variance is not concentrated in a small number of embedding directions.

therefore sparse relative to all pairs. Figure 3 illustrates this empirical pattern for IMDB actor–director embeddings: most distances lie between 0.55 and 0.85, with only a small close-pair tail.

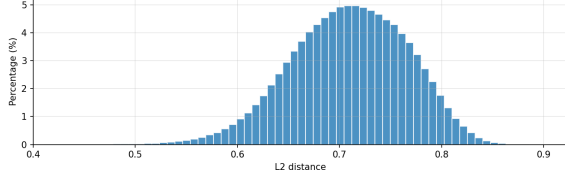


Figure 3: L2 distance distribution: pairwise among IMDB Actor_Director embeddings (Gemini-embedding-001)

This observation motivates materializing the close-pair tail as a **sparse embedding-similarity join index** and feeding its endpoints to batched retrieval. (Real endpoint populations can be anisotropic and their distance distributions need not be Gaussian, but still exhibit the tail property; Section 3.8 discusses mitigations.)

3 Semantic Join Index

Based on the intuition from the previous discussion, the **Semantic Join Index (SEMJI)** materializes close vector pairs, instantiating the embedding-similarity-join topology $\mathcal{J}$ of our query model and avoiding repeated join discovery at query time. Here “semantic” refers to the embedding-based proxy; it is distinct from a downstream LLM semantic operator.

3.1 Motivation and Definition

For materialization radius τ_{cap} , we materialize relation $\mathcal{J}_{\tau_{cap}} = \{(A, B) \mid \text{dist}(A, B) \leq \tau_{cap}\}$. Each entry $(id_A, tid_A, id_B, tid_B, \text{dist}(A, B))$ records the source fields, tuple pointers, and distance; the distance is converted to the query’s higher-is-better affinity only during ranking (e.g., by subtracting from 1). Thus, τ_{cap} is an **index-coverage boundary**. Standard relational joins chain multiple SEMJI steps for multi-hop patterns; this composes thresholded links and does not assume transitive embedding similarity. (We discuss correlation in Section 3.8.)

3.2 Correctness and Coverage

An exact SEMJI construction stores every pair in $\mathcal{J}_{\tau_{cap}}$. For a query threshold $\tau_q \leq \tau_{cap}$, the index answers its similarity-join predicate exactly. For $\tau_q > \tau_{cap}$, exact execution additionally enumerates the *outer band* $\tau_{cap} < \text{dist}(A, B) \leq \tau_q$ through the base engine.

An ANN-built SEMJI is instead approximate: it stores $\hat{\mathcal{J}}_{\tau_{cap}} \subseteq \mathcal{J}_{\tau_{cap}}$. Filtering by distance removes false positives among retrieved candidates, but ANN traversal can miss an under-the-cap pair and hence create false negatives. Completing only the outer band cannot repair such a miss. Therefore, a result using an approximate SEMJI is exact only with respect to its retrieved candidate set. The approximate SEMJI inherits any guarantee of its underlying ANN method; HNSW has no general completeness guarantee, whereas some alternatives have approximation bounds [16].

3.3 Index Construction

The SEMJI construction process determines what guarantees it provides.

Exact construction. Exact construction enumerates the Cartesian product and retains every pair within τ_{cap} ; it provides the ground truth for approximate-construction recall.

Approximate construction. leverages an ANN structure. For a neighbor relationship, it probes one table against an index over the second one, and exactly refines all returned candidates. This filter-and-refine procedure removes returned candidates outside τ_{cap} but does not establish coverage.

E2LSH filter-and-refine (alternative strategy). For L_2 distance, E2LSH [?] hashes each vector x using $h_{a,b}(x) = \lfloor (a^\top x + b)/w \rfloor$, where $a \sim \mathcal{N}(0, I)$ and $b \sim \text{Uniform}[0, w]$. The build phase stores each larger-relation vector in one or more hash tables; the probe phase retrieves larger-relation vectors sharing a hash bucket with each smaller-relation vector. We then compute exact distances for these collision candidates and materialize only pairs satisfying $\text{dist}(a, b) \leq \tau_{cap}$. Thus, refinement eliminates false positives, whereas missed collisions may yield false negatives. We did not implement this strategy in practice, since a hash-table and bucket-management path is less well suited to an in-engine PostgreSQL implementation than HNSW traversal. If enabled, it requires reporting the number of tables, concatenated hashes, and tuned width (for example, $w \approx 4\tau_{cap}$ is only a high-recall heuristic).

Radius-expanded HNSW (Algorithm 1), preferred strategy. Standard HNSW is optimized for top- k retrieval and can prune candidates inside a requested radius. Our **radius-expanded traversal** relaxes the pruning gate to the maximum of the beam boundary and τ_{cap} : $\text{gate} = \max(W.\text{furthest}(), \tau_{cap})$. This improves empirical range-query recall but does not establish completeness; we measure it against exact construction.

3.4 Space Cost

SEMJI is a materialized relation, with $J = |\mathcal{J}_{\tau_{cap}}|$ retained pairs. Its base representation and any distance/order or endpoint lookup indexes require $O(J)$ records. Because the SEMJI is constructed only for frequently used embedding-column pairs whose calibrated tail has a bounded expected degree, the retained-pair fraction $\rho = J/(|A||B|)$ is small, so storage scales as $\rho|A||B|$ rather than the full product. Nonetheless, we enforce a maximum space budget per index, adjusting τ_{cap} to reach that budget. The configurations in Section 5 show practical settings.

3.5 Batched Execution Optimization

Both exact and approximate SEMJI construction options batch probes. Exact construction uses blocked, vectorized distance computation and writes only surviving pairs. Approximate construction groups HNSW probes with shared upper-layer traversals and E2LSH probes by hash bucket, improving locality for candidate access.

3.6 Multi-Step Reasoning

SEMJI supports multi-hop paths (e.g., $A \xrightarrow{\tau_1} B \xrightarrow{\tau_2} C$) by treating its materialized connectivity as a relational view. For a chain $\mathcal{J}_{chain} = \{(\mathcal{J}_1, \tau_1), \dots, (\mathcal{J}_m, \tau_m)\}$, the engine computes:

$$\mathcal{R}_{chain} \leftarrow \text{Fetch}(\mathcal{J}_1, \tau_1) \bowtie \text{Fetch}(\mathcal{J}_2, \tau_2) \bowtie \dots \bowtie \text{Fetch}(\mathcal{J}_m, \tau_m) \quad (1)$$

Algorithm 1 Layer Traversal in Radius-Expanded HNSW

Require: Query q , Entry Point ep , Beam ef , Query Radius $\tau_q \leq \tau_{cap}$, Construction Cap τ_{cap}

Ensure: Approximate Result Set $\mathcal{R}$

```

1:  $d_{ep} \leftarrow \text{dist}(ep, q)$ ;  $C \leftarrow \text{MinHeap}(ep, d_{ep})$ ;  $\mathcal{W} \leftarrow \text{MaxHeap}(ep, d_{ep})$ ;  $\mathcal{V} \leftarrow \{ep\}$ 
2:  $\mathcal{R} \leftarrow \emptyset$ 
3: if  $d_{ep} \leq \tau_q$  then  $\mathcal{R}.\text{add}(ep)$ 
4: end if
5: while  $C$  is not empty do
6:    $c \leftarrow C.\text{pop}()$ ;  $w \leftarrow \mathcal{W}.\text{peek}()$ 
7:   if  $c.\text{dist} > w.\text{dist}$  and  $c.\text{dist} > \tau_{cap}$  then
8:     break  $\triangleright$  Prune only if outside both beam and radius
9:   end if
10:   $gate \leftarrow \max(\mathcal{W}.\text{peek}().\text{dist}, \tau_{cap})$ 
11:  for  $e \in \text{neighbors}(c)$  do
12:    if  $e \notin \mathcal{V}$  then
13:       $\mathcal{V}.\text{add}(e)$ ;  $d_e \leftarrow \text{dist}(e, q)$ 
14:      if  $d_e \leq gate$  or  $|\mathcal{W}| < ef$  then
15:         $C.\text{push}(e, d_e)$ ;  $\mathcal{W}.\text{push}(e, d_e)$ 
16:        if  $|\mathcal{W}| > ef$  then  $\mathcal{W}.\text{pop\_furthest}()$ 
17:      end if
18:      if  $d_e \leq \tau_q$  then  $\mathcal{R}.\text{add}(e)$ 
19:    end if
20:  end if
21: end for
22: end while
23: return  $\mathcal{R}$ 

```

where each *Fetch* applies its runtime threshold $\tau_i \leq \tau_{cap}$. Projecting $\mathcal{R}_{chain}$ onto a target table yields the semantic semijoin predicate $P_{\mathcal{J}}$, which the Filtered-Score Streamer (Section 4.2) pushes down before scoring.

3.7 Capacity-Based Threshold Maintenance

We can maintain both exact and approximate SEMJI structures with the same basic mechanism. Initially, the SEMJI has a τ_{cap} set based on a small warmup sample and query. Of course, the workload may introduce queries in the future that require a different threshold. Thus, the controller grows the index by its *materialized-pair capacity*, which allows us to limit the estimated number of growth steps. Let $J(\tau) = |\mathcal{J}_\tau|$ and let $J = J(\tau_{cap})$ be the current number of retained pairs. A space budget B_{max} bounds every SEMJI.

The budget is on retained pairs, not on the numerical radius: embedding geometry can make a small change in radius admit many pairs, whereas J is directly proportional to base SEMJI storage (up to fixed per-pair index overhead). Even with a large budget, we would expect to materialize a very small percentage of the possible pairs, as we see in Section 5.

Overflow and capacity doubling. When a query requests $\tau_q > \tau_{cap}$, DASE answers it both using the SEMJI along with base-engine enumeration of the outer band $\tau_{cap} < \text{dist}(a, b) \leq \tau_q$. Thus, index growth is not on the query’s critical path. The controller records the overflow and asynchronously doubles its retained-pair capacity,

targeting B' pairs at radius τ_{next} ,

$$B' = \min\{B_{max}, \max\{2J, J_{min}\}\}, \quad \tau_{next} = \inf\{\tau \geq \tau_{cap} : J(\tau) \geq B'\},$$

where J_{min} prevents a near-empty initial index from growing in negligible increments. The controller expands only when $J < B_{max}$, materializes the new values in the range $(\tau_{cap}, \tau_{next}]$, and publishes a new index version. A single unusually permissive query need not make the index cover τ_q : strict capacity doubling preserves the budget, while that query still uses its fallback. Ties or a sharp density increase can make $J(\tau_{next})$ exceed B' ; the controller rejects that expansion if it would exceed B_{max} .

This policy is independent of the distribution of pairs under the threshold, although anisotropy can change the radius needed to admit B' pairs. We additionally monitor endpoint-degree skew, since a modest total J concentrated at a few endpoints can still make downstream joins expensive.

Amortization and data evolution. For a static relation, capacity doubling yields at most $\lceil \log_2(B_{max}/J_{min}) \rceil$ expansions. If an expansion discovers, writes, and indexes only new distance-bands, the total pair-materialization work through final size J_f is $O(J_f - J_0)$; even rebuilding the materialized relation at every doubling has a geometric-series bound below $2J_f$ for pair writing. This is an output-space amortization guarantee, where the cost of the rescan is dependent on the radius-expanded HNSW of Algorithm 1.

3.8 Robustness to Correlated Embeddings

Our SEMJI materialization-radius heuristic relies on the pairwise-distance population being sufficiently isotropic for a tail threshold such as $\tau_{cap} = \mu - 3\sigma$ to be informative. This can fail when endpoint embeddings are anisotropic or share dominant directions, because they were extracted from overlapping source context. Such geometry can distort the apparent distance distribution, so a statistically chosen materialization radius may omit useful close pairs [11, 28, 38]. We can detect anisotropy by computing the covariance of the centered embeddings from a query; if the variance is concentrated in a few directions, this indicates that a raw-distance tail estimate may be error-prone once predicates or other neighborhood relationships are factored in.

In lieu of the baseline strategy of calibrating thresholds by $\mu - 3\sigma$ over a sample, SEMJI can adopt mitigation strategies from the literature for anisotropic data. **PCA whitening** mean-centers and rescales principal axes [38]; **all-but-the-top (ABTT)** removes a configured number of dominant directions [28]. Because these transformations define different retrieval metrics, SEMJI recalibrates τ_{cap} and builds a matching transformed index; it never queries transformed vectors against the raw index.

Even when anisotropy makes the choice of the index-selection threshold unreliable, it need not make the admitted relation dense. SEMJI materializes pairs with $\text{dist}(a, b) \leq \tau_{cap}$; equivalently, τ_{cap} retains pairs whose distance-derived affinity is at least the calibrated threshold. As Section 5.7 shows, these near-neighbor edges remain a tail of the real endpoint-pair products (0.03–1.09% under the evaluated policies), even for the anisotropic corpora.

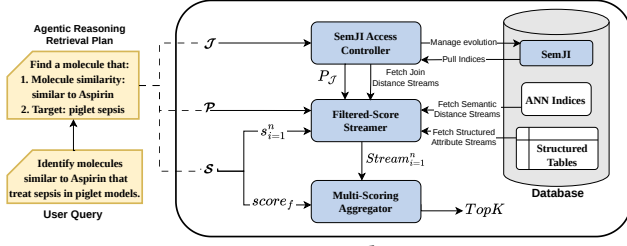


Figure 4: DASE architecture

4 DASE Execution Strategy

The execution layer of DASE consists of three core components that interoperate to tackle multi-step reasoning with both similarity and Boolean predicates: the **Multi-Scoring Aggregator** (MSA), multiple **Filtered-Score Streamers** (FSSs), and the **SEMJI Access Controller**, which exposes materialized connectivity. Predicate-aware ANN traversal is one access mode within FSS, alongside predicate-first and vector-first execution. Our approach requires a coordinatewise non-decreasing scoring function over higher-is-better component scores [12].

4.1 Predicate-aware ANN Index Traversal

Most hybrid search systems split their strategies into two camps [16]: *vector-first* (post-filtering) and *filter-first* (pre-filtering). This creates a performance gap: vector-first works well when most items match the filter, while filter-first is better when the filter eliminates almost everything. Between these two extremes lies a “middle ground” where neither approach is efficient. To handle this, we implemented a **Predicate-aware ANN Index Traversal** that checks filters while walking the index graph. **Often one of the filter conditions is whether the ANN node endpoints lie within a batch of path endpoints retrieved from the SEMJI index.**

Simulating an Oracle Graph. Our method is inspired by the *Oracle Graph* idea from ACORN [31]. Conceptually, if we had a custom index built only from the items that pass the filter, search would be optimal. Since we cannot build a new index for every query, we simulate this experience using the global HNSW index.

Transitive neighbor expansion. The main problem with walking a global ANN graph under constraints is that a path to a good candidate may need to traverse a node that fails the filter. To mitigate this, we modify the greedy search: When the immediate neighbors of a node are all filtered out, we look at their neighbors (2 hops away). By using these invalid nodes as bridges rather than dead ends, we can jump over gaps in the graph and find the true nearest neighbors that satisfy the query.

Bloom filter optimization. As noted above, DASE will frequently combine SEMJI results with an ANN traversal — doing a similarity join over a similarity-based lookup. Checking the filter condition for every node during a graph traversal is too slow if it requires fetching data from the main table. To speed this up, we compile the query’s node filter set into a **Bloom filter** [3] before the search begins. This allows the traversal engine to check if a node is valid using a fast bitwise operation. For a target false-positive rate ϵ , the Bloom filter uses $O(N \log(1/\epsilon))$ bits and takes $O(N)$ time to build. If the dataset is too large, the system falls back to standard disk-based strategies to avoid exhausting memory.

4.2 Filtered-Score Streamer

The **Filtered-Score Streamer** (FSS) generates a sorted stream for one higher-is-better scoring signal under constraints. Formally, its input is (1) a set of predicates $\mathcal{P}$ and (2) a scoring signal s , which is one of: (a) query-to-object affinity, (b) relational-link affinity, or (c) structural-attribute desirability, according to the Query Model in Section 2. Distance-derived signals are first transformed by the query’s non-increasing affinity function. The output is an iterable stream of entries that pass the filters in descending score order: $FSS(\mathcal{P}, s) \rightarrow Stream((id_i, score_i))$. The FSS adapts its execution path based on the data source and predicate selectivity σ .

(a) Semantic Path: For a query-to-object affinity derived from $dist(\mathbf{q}, \mathbf{e}_i)$, the scorer selects between attribute-first filtering, vector-first ANN traversal, and the predicate-aware ANN traversal based on σ to minimize distance-computation overhead. When $\sigma < \sigma_1$, attribute-first filtering leaves a short candidate list for direct distance computation; when $\sigma > \sigma_2$, vector-first search with expanded retrieval and post-filtering is preferable; and when $\sigma_1 < \sigma < \sigma_2$, the system uses predicate-aware traversal.

Selection of σ thresholds. Offline profiling identifies crossover selectivities σ_1, σ_2 at a fixed recall target: attribute-first below σ_1 , vector-first above σ_2 , and predicate-aware ANN between them.

(b) Relational Path: For a relational-link affinity derived from $dist(A, B)$, the scorer scans the **join index** in ascending distance order (equivalently, descending affinity order), while applying $\mathcal{P}$ to filtering attributes stored in the join index or entity metadata. This can be accelerated by traditional B-tree indices on the join index.

(c) Attribute Path: For a structured attribute scoring signal, we perform filtering with B-tree indices over the relational data.

4.3 Multi-Scoring Aggregator

The **Multi-Scoring Aggregator** (MSA) combines the sorted streams $\{S_i\}_{i=1}^n$ produced by FSS under a coordinatewise non-decreasing, higher-is-better aggregation function $score_f$, returning $MSA(\{S_i\}_{i=1}^n, score_f, K)$. It is a direct adaptation of Fagin’s Threshold Algorithm (TA) [12], with Rank-Join-style random access [24] to reconstruct relationally valid tuples. Thus, MSA does not replace TA’s ranking logic: it retains sorted access, random access, a top- k buffer, and TA’s threshold stopping test.

On observing an entry e from any stream, MSA uses SEMJI lookups and sibling attributes to materialize every valid tuple containing e , scores each newly discovered tuple, and maintains the best K . The only DASE-specific changes are that random access can traverse a multi-hop SEMJI predicate and that the next stream need not be selected round-robin. If $L[i]$ is the most recently seen score of stream i , every unmaterialized tuple lies below every current cursor and is bounded by $\tau = score_f(L[1], \dots, L[n])$. MSA stops when its current K -th score is at least τ .

Exact answers. This is the usual TA guarantee: the result is exact when all streams are complete and globally sorted, random access reconstructs every valid tuple containing a seen entry, and component scores are exact. If we leverage ANN traversal or an approximate SEMJI, the stopping rule provides exact answers only with respect to the set of retrieved candidates.

Distribution-Aware Stream Selection. TA normally advances streams in round-robin order. MSA instead chooses the stream expected to reduce the termination gap $G = \tau - \theta_K$ most, where θ_K is the current K -th score. For a linear $score_f = \sum_i w_i s_i$, profiling supplies each stream’s local score density f_i (an empirical density or a fitted Gaussian). The expected threshold reduction from advancing stream i is approximately $w_i / (N f_i(L[i]))$. MSA also estimates the expected increase in θ_K from the newly materialized tuples using the other streams’ cursor-truncated score distributions, and advances

$$i^* = \arg \max_i \{E[\Delta \tau_i] + E[\Delta \theta_{K,i}]\}.$$

Density and truncated-expectation values are tabulated during profiling. This is a greedy scheduling policy layered on TA, without changing its stopping condition or exactness guarantees. Its candidate-elevation estimate uses marginal stream distributions, and is therefore heuristic when ranking signals are correlated.

4.4 SEMJI Access Controller

The **SEMJI Access Controller** functions as the storage abstraction layer that exposes the materialized semantic connectivity of SEMJI to the query pipeline. It serves two distinct query-time roles depending on whether the relational data is treated as a hard constraint or a ranking signal. Additionally, it acts as the runtime monitor that drives the structural evolution of the index.

Materializing join predicates. When the query includes hard relational constraints ($\mathcal{J}$), the controller’s objective is to define the valid search space. It retrieves the set of materialized pairs for each join condition and, in the case of multi-hop dependencies, joins these sets to synthesize a unified **candidate set** of valid tuples. This process effectively converts topological constraints into a flat filter predicate $P_{\mathcal{J}}$, which is pushed down to the *Filtered-Score Streamer* to prune the search space. Symbolically, $P_{\mathcal{J}} \leftarrow \bigwedge_{i=1}^{|\mathcal{J}|} \text{SemJIFetch}(\mathcal{J}_i)$.

Streaming relational scores. When the query utilizes relational proximity as a ranking objective (i.e., a scoring component $s \in \mathcal{S}$ is defined by the semantic link distance), the controller acts as a streaming source. It scans the underlying index to yield pairs (id, dis) sorted by their precomputed semantic distance. This stream is fed directly into the *Multi-Scoring Aggregator* as a standard ranking signal: $S_{rel} \leftarrow \text{SemJIFetch}(\mathcal{J}_s)$.

Workload monitoring and evolution trigger. As described in Section 3.7, the controller performs workload monitoring. During query execution, it tracks the *base-engine completion rate*: the fraction of queries whose materialized SEMJI coverage is exhausted, i.e., that overflow to $\tau_q > \tau_{cap}$. If this rate consistently exceeds its tolerance, the controller asynchronously triggers a capacity-doubling expansion, growing $\tau_{cap} \rightarrow \tau_{next}$ within the budget B_{max} via Radius-Expanded HNSW. This allows the index to evolve without bottlenecking real-time execution.

5 Experiments

Datasets. To evaluate DASE on deep research reasoning tasks, we constructed two benchmark datasets. In **IMDB**, base tables from the official IMDB (<https://datasets.imdbws.com>) are augmented with structured fields from the repository `jquig1/imdb-genres`. To support cross-table movie comparisons, the data is split by release year. In **MOLECULE**, we collected query templates from our

agentic AI system built on a PubMed-derived molecule database (<https://pubmed.ncbi.nlm.nih.gov/>). These templates are used to simulate multi-step reasoning workflows (e.g., discovering candidate molecules by joining fragmented evidence). The dataset consists of Papers, Facts, and Molecules. Benchmark and workload details are available in our technical report (<https://anonymous.4open.science/r/DASE-2C61/>).

We include two public benchmark datasets that allow us to examine specific aspects of DASE. The first of these is **SemBench** [23], which we use to explore the effectiveness of DASE in prefiltering for semantic queries. Our second benchmark is **NFCorpus** [4], a 2,868-paper biomedical corpus with title embeddings, real intervention-condition and study-outcome links, and categorical attributes. Its depth-scaling workload ranges from two to eight join steps, providing diverse multi-step reasoning queries at different cardinalities.

Workloads. We developed synthetic workloads over our two datasets, following a taxonomy shown in Table 2 and fully detailed in our technical report. These cover all possible combinations of structured predicates ($\mathcal{P}$), multi-component scoring ($\mathcal{S}$), and cross-table joins ($\mathcal{J}$). In this framework, single-vector similarity search serves as the default configuration. The component $\mathcal{S}$ represents the transition from simple embedding retrieval to a generalized scoring function $f(s_1, \dots, s_n)$. This allows the system to aggregate multiple vector similarities, as required for complex discovery workflows.

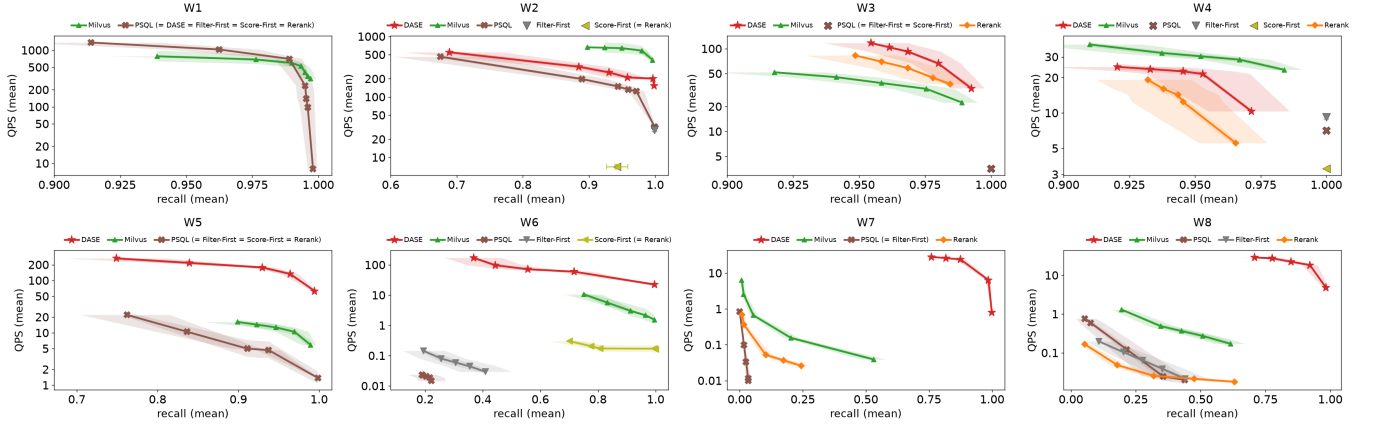
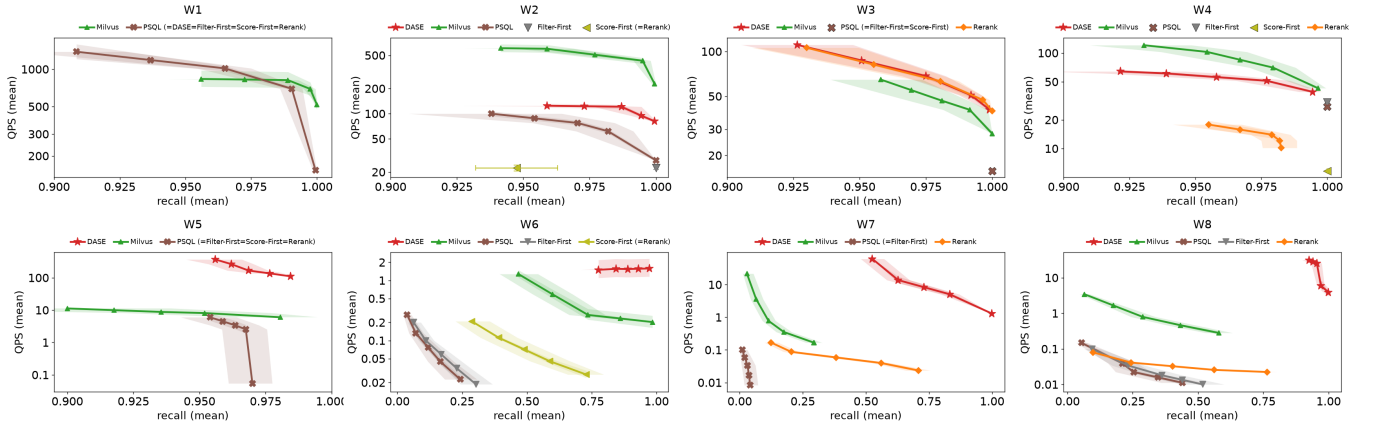
Additional workloads are from the published SemBench and NFCorpus benchmark specifications.

Implementation strategies. In deep research tasks, we compare DASE against five distinct execution strategies and DBMSs:

- (1) **PostgreSQL:** The multi-step reasoning query is submitted directly to the PostgreSQL engine. The embedding columns are indexed using pgvector’s HNSW or IVFFlat structures. All execution decisions, including join order, index selection, and predicate pushdown, are delegated to the PostgreSQL optimizer.
- (2) **Filter-First:** Evaluates structural and exact-match predicates first (using the WHERE clause) to generate a candidate set of valid IDs. The system then performs random access to fetch the corresponding embeddings, computes the similarity scores on-the-fly, and maintains a Top- k heap to produce the result.
- (3) **Score-First:** An inverted strategy that prioritizes similarity search. It uses an ORDER BY clause to retrieve a continuous stream of records sorted by their joint similarity score. The system applies the structural predicates as a post-filter and halts the stream immediately once k valid entities are identified.
- (4) **Rerank:** For queries involving multiple scoring signals, executes a retrieve-and-rerank pipeline. It streams candidates separately from each signal’s index, applies predicates as post-filters, and dynamically evaluates the joint score to update the Top- k results until the exploration stopping criteria are met. For single-signal queries, it gracefully degrades to Score-First.
- (5) **Milvus:** Milvus [42] is a state-of-the-art vector database. For single-signal queries (with or without structural predicates), we push down the filters directly into Milvus. Because Milvus lacks support for multi-signal scoring or relational joins, queries involving these operations are streamed to the application layer, which applies the Rerank strategy to finalize the Top- k results.

Table 1: Main experimental data and configuration. SEMJI counts are the materialized-pair records in the released configurations; a workload selects the radius appropriate to its join threshold.

Scenario	Base relations	Candidate Links / SEMJI pairs	Workload (queries/class, top-20)
IMDB	imdb_t1: 42,378 rows; imdb_t2: 43,237 rows	$\tau_{cap} = 0.5$: 2.10M; 0.6: 58.41M pairs	W1–W8; 84–100
MOLECULE	facts_50k: 50,000 rows; paper: 62,857 rows	$\tau_{cap} = 0.5$: 0.032M; 0.6: 1.38M; 0.7: 63.48M pairs	W1–W8; 93–100
SemBench	Movie, E-Commerce, Wildlife, MMQA, and Cars scenarios	Calibrated embedding candidate links for semantic joins; no shared deep-research SEMJI	F/L/J/M/C/R tasks; DASE prefilter and DASE + BigQuery
NFCorpus	2,868 papers; title embeddings	$\tau = 0.35$: 21,466 intervention-condition; 254 study-outcome pairs	depth 2–8; 90 instances (3×30)

**Figure 5: Multi-step Reasoning Top-k: Deep Research workload on IMDB****Figure 6: Multi-step Reasoning Top-k: Deep Research workload on MOLECULE**

5.1 Setup and Methodology

DASE is implemented in about 10,000 lines of Python and C, with part of the development driven by Claude Code (Opus 4.6–4.8) and extensive manual code review. Experiments were conducted on PostgreSQL 18.3 with the DASE-modified pgvector extension (v0.8.5). Text vectors use Gemini-embedding-001 (1536 dimensions); MOLECULE additionally includes molecular descriptors and an ECFP signal. We evaluated a set of multi-step reasoning queries involving structured filters, multi-vector relevance scoring, and join-based conditions. All vector columns use L2 HNSW indexes with $m = 16$, $ef_{construction} = 64$, and default $ef_{search} = 40$. Each database instance runs on a dedicated set of 16 physical AMD EPYC 9575F cores (up to 5.0 GHz) and 64 GB of RAM. This hardware supports

AVX-512 instructions, which pgvector leverages to accelerate vector distance computations. Evaluation was CPU-based; no GPUs were used.

We report each method as a recall-throughput curve. For every method we sweep a search-effort parameter and record recall against QPS: the HNSW ef_{search} and IVFFlat probe count for index scans, the threshold-algorithm admission tolerance ϵ for DASE and Rerank, the post-filter width for Milvus, and a per-query time budget for best-effort strategies. Each operating point averages 5 runs after discarding a warm-up run, and shaded bands denote ± 1 standard deviation; per method we retain the Pareto frontier over the resulting (recall, QPS) points. Recall is measured against the released exhaustive top-20 result for each planned query. Index construction time is isolated and excluded from all QPS measurements.

Table 2: Characterization of query workload classes.

WL	$\mathcal{P}$	Multi- $\mathcal{S}$	$\mathcal{J}$	Pattern	Expression
<i>Intra-Entity Retrieval</i>					
W_1	×	×	×	Semantic retrieval	$TopK\{FSS(\emptyset, s)\}$
W_2	✓	×	×	Filtered retrieval	$TopK\{FSS(\mathcal{P}, s)\}$
W_3	×	✓	×	Multi-scoring retrieval	$MSA(\{FSS(\emptyset, s_i)\}_{i=1}^n, score_f, K)$
W_4	✓	✓	×	Filtered multi-scoring	$MSA(\{FSS(\mathcal{P}, s_i)\}_{i=1}^n, score_f, K)$
<i>Inter-Entity Synthesis</i>					
W_5	×	×	✓	Semantic join	$TopK\{FSS(P_{\mathcal{J}}, s)\}$
W_6	✓	×	✓	Filtered join	$TopK\{FSS(\mathcal{P} \cup P_{\mathcal{J}}, s)\}$
W_7	×	✓	✓	Multi-scoring join	$MSA(\{FSS(P_{\mathcal{J}}, s_i)\}_{i=1}^n, score_f, K)$
W_8	✓	✓	✓	Filtered multi-scoring join	$MSA(\{FSS(\mathcal{P} \cup P_{\mathcal{J}}, s_i)\}_{i=1}^n, score_f, K)$

Table 1 reports the dataset-specific relation, SEMJI, and workload configurations; query-specific thresholds and score weights are stored with the released workload instances.

For SemBench, **DASE** performs ranked embedding-based pre-filtering, omitting LLM judgment, using counterfactual anchors for semantic filters and calibrated thresholds for semantic joins. **DASE + BigQuery** sends the same candidates to BigQuery semantic operators; its cost and latency therefore describe the composed pipeline. Table 3 reports USD, the SemBench task metric, and end-to-end wall-clock latency including remote Gemini API time; X denotes unsupported configurations and scenario averages exclude them. The LOTUS, Palimpzest, ThalamusDB, and BigQuery columns reproduce SemBench’s published gemini-2.5-flash configuration [23]; only the DASE and DASE + BigQuery columns are ours.

5.2 Multi-Step Top- k Reasoning

Deep Research. We evaluate DASE and the alternatives across the **IMDB** (Figure 5) and **MOLECULE** (Figure 6) datasets over our 8 workloads. Each panel plots the recall-throughput (QPS) Pareto frontier obtained by sweeping each method’s search-effort as introduced in the setup section (Section 5.1); up and to the right is better. Milvus excels on the single-signal W1 workload, but its advantage reverses when queries require multi-score ranking. Filter-first becomes a sequential scan without predicates, rerank reduces to score-first with one signal, and score-first is infeasible for W7/W8 because it forms a full Cartesian product. On W3, DASE outperforms round-robin rerank, indicating the value of adaptive stream selection; across W5–W8, DASE and SEMJI sustain higher QPS at matched recall. We separately study how performance scales with the number of coupled join positions in Section 5.4.

Semantic queries. Table 3 summarizes cost, quality, and latency on SemBench. DASE alone suffices for semantic-filter (**F**) tasks that reduce to a binary or coarse distinction, such as positive vs. negative reviews in *Movie*: comparing embedding distance to counterfactual anchors returns valid answers at orders-of-magnitude lower cost and latency than any LLM-backed baseline. For semantic-map (**M**) and LLM-semantic-join (**J**) tasks that exceed embedding-distance reasoning, DASE + BigQuery substantially raises quality (e.g., *MMQA* avg. 0.11 $\rightarrow$ 0.76). Prefiltering with DASE also lets BigQuery beat its own standalone numbers at a fraction of the cost: on *E-Commerce*, \$0.54/38s at quality 0.80 versus BigQuery alone at

\$2.42/45s and 0.67. The LLM sees only the prefilter’s survivors, so it spends less and decides better. The one failure mode is embedding quality itself: in *Wildlife*, rare words like “impala” are poorly aligned, so neither configuration recovers fully.

Overall, DASE delivers low-cost answers when embedding distance suffices, and a high-recall prefilter that makes downstream LLM evaluation more efficient and accurate when it does not.

5.3 SEMJI Construction and Maintenance

We build SEMJI using HNSW-based ANN range queries with a radius threshold. Table 4 shows an ablation from a naive iterative approach, through a batched lateral join, to a fully vectorized whole-table HNSW range query — a single JOIN LATERAL that runs entirely in-database. This reduces construction time from 81.2 hours to 1.6 minutes, only losing 2% from recall.

SEMJI evolution. We evaluate the capacity-based maintenance policy (Section 3.7) on the linkage-only MOLECULE W5 stream of 2,000 seed queries. A random warmup sample fixes $\tau_{init} = \mu - 3\sigma = 0.607$ before the workload is observed; overflowing queries use the base-engine outer-band fallback while the controller grows the index asynchronously.

On this stream, two capacity doublings grow SEMJI from 83K to 333K pairs and reach $\tau_{converged} = 0.6526$ (Figure 7). The final 333K retained pairs are only 0.011% of the 3.14B fact-paper Cartesian product (6.7 links/fact and 5.3 links/paper on average), so SEMJI remains highly compact relative to exact join enumeration. The held-out overflow rate falls from 80.6% to 49.8%; the remaining overflows are budget-limited sparse seeds. Ascending and random query orders reach the same final radius (spread < 0.001) with two expansions, differing only in when the work is paid (a 1.14× cumulative-latency difference).

5.4 Scaling Multi-step Reasoning Depth

We isolate how each system scales vs. the number of coupled join positions k , on the multi-dimensional NFCORPUS workload $Q = (P, S, J)$. Here a k -step query fixes per-position anchors $a_1, \dots, a_k$ and asks for the top- n length- k walks $(p_1, \dots, p_k)$ satisfying

$$\min \frac{1}{k} \sum_{i=1}^k \text{dist}(a_i, p_i) \quad \text{s.t. } P(p_i) \forall i, \text{dist}(p_i, p_{i+1}) \leq \tau \forall i, \quad p_i \neq p_{i-2},$$

i.e. a monotone score S (title-embedding distance) minimized subject to a predicate P (selectivity $\sigma=0.18$) and a threshold intervention-condition similarity join J ($\tau=0.35$) between consecutive positions, without immediate backtracking. We sweep $k=2 \dots 8$ and report recall against the exact top- n result and throughput (Figure 8; 95% CI over 90 instances, 3 seeds $\times$ 30).

POSTGRESQL hands the whole predicate-filtered k -way self-join to the SQL planner: it uses the materialized edge relation, but produces a combinatorial intermediate. MILVUS can apply per-position scalar filters, but cannot execute the cross-position join J or rank the paths. It therefore retrieves the per-position top- L nearest neighbors, applies P , and reranks the surviving length- k walks in the application layer (L is its accuracy knob). Given the materialized SEMJI edge set, DASE recognizes the query as a *minimum-cost bounded-length walk* on the join graph and uses a Viterbi-like dynamic program with beam width n (the number of returned walks).

Table 3: SemBench cost, quality, and latency (notation: F = filter; L = limit- k ; J = join; M = semantic map; C = semantic classify; R = semantic rank)

	LOTUS [30]			Palimpzest [26]			ThalamusDB [20]			BigQuery [13]			DASE			DASE + BigQuery		
	Cost	Qual.	Lat.	Cost	Qual.	Lat.	Cost	Qual.	Lat.	Cost	Qual.	Lat.	Cost	Qual.	Lat.	Cost	Qual.	Lat.
(a) Movie Scenario										DASE Preprocess			\$3·10 ⁻³		27.6 s	\$3·10 ⁻³		27.6 s
Q1: F L	\$0.09	1.00	33.1 s	\$1·10 ⁻³	1.00	3.8 s	\$4·10 ⁻⁴	0.95	4.2 s	\$0.05	1.00	26.3 s	\$2.4·10 ⁻⁶	1.00	0.5 s	\$1·10 ⁻⁴	1.00	3.8 s
Q2: F L	\$0.01	1.00	2.1 s	\$0.01	1.00	29.7 s	\$2·10 ⁻³	0.92	1.9 s	\$3·10 ⁻³	1.00	9.5 s	\$2.4·10 ⁻⁶	1.00	0.5 s	\$1·10 ⁻⁴	1.00	4.2 s
Q3: F	\$5·10 ⁻³	0.64	2.1 s	\$0.01	0.64	4.6 s	\$2·10 ⁻³	0.74	3.1 s	\$3·10 ⁻³	0.64	11.0 s	\$3.6·10 ⁻⁶	0.82	0.5 s	\$7·10 ⁻⁴	0.82	6.2 s
Q4: F	\$5·10 ⁻³	0.64	2.8 s	\$0.02	0.74	4.4 s	\$2·10 ⁻³	0.74	3.8 s	\$3·10 ⁻³	0.64	11.4 s	\$3.6·10 ⁻⁶	0.82	0.5 s	\$7·10 ⁻⁴	0.82	5.3 s
Q5: J L	\$2.38	0.59	536.5 s	\$0.01	0.39	1.9 s	\$1·10 ⁻³	1.00	2.3 s	\$1.01	0.89	54.5 s	\$3.6·10 ⁻⁶	1.00	0.5 s	\$6·10 ⁻⁴	0.82	3.9 s
Q6: J L	\$1.81	0.67	432.4 s	\$0.01	0.83	2.3 s	\$9·10 ⁻⁴	0.84	1.7 s	\$1.00	0.69	54.5 s	\$3.6·10 ⁻⁶	1.00	0.5 s	\$7·10 ⁻⁴	0.94	3.8 s
Q7: J	\$1.81	0.21	431.8 s	\$7.72	0.68	1056.1 s	\$0.15	0.57	636.9 s	\$3.31	0.70	198.3 s	\$3.6·10 ⁻⁶	0.73	0.5 s	\$9·10 ⁻⁴	0.71	3.8 s
Q8: C	\$4·10 ⁻³	0.93	2.3 s	\$0.02	0.86	4.3 s	\$5·10 ⁻³	0.83	6.8 s	\$3·10 ⁻³	0.76	10.9 s	\$3.6·10 ⁻⁶	0.89	0.5 s	\$7·10 ⁻⁴	0.89	4.2 s
Q9: R	\$0.02	0.75	4.9 s	\$0.05	0.78	5.7 s	X	X	X	\$0.02	0.78	13.3 s	\$4.8·10 ⁻⁶	0.66	0.5 s	\$6·10 ⁻³	0.74	12.8 s
Q10: R	\$0.13	0.40	30.9 s	\$0.38	0.42	39.2 s	X	X	X	\$0.13	0.44	32.1 s	\$4.8·10 ⁻⁶	0.43	0.5 s	\$0.085	0.41	26.1 s
Avg	\$0.626	0.68	147.9 s	\$0.823	0.73	115.2 s	\$0.02	0.82	82.6 s	\$0.553	0.75	42.2 s	\$3·10 ⁻⁴	0.83	3.0 s	\$9·10 ⁻³	0.81	9.2 s
(b) E-Commerce Scenario										DASE Preprocess			\$0.03		124.0 s	\$0.03		124.0 s
Q1: F	\$0.06	1.00	12.2 s	\$0.08	1.00	12.2 s	\$0.03	1.00	34.8 s	\$0.04	0.59	21.2 s	\$5·10 ⁻⁶	0.94	0.7 s	\$7·10 ⁻³	1.00	8.0 s
Q2: F	\$0.18	0.87	166.1 s	\$0.38	0.83	57.5 s	\$0.31	0.67	100.8 s	\$3.96	0.21	55.7 s	\$7·10 ⁻⁶	0.47	0.7 s	\$0.02	0.66	5.5 s
Q3: M	\$0.07	0.97	17.1 s	\$0.12	0.98	16.5 s	X	X	X	\$0.12	0.97	21.2 s	X	X	X	\$0.04	0.93	10.5 s
Q4: M	\$0.14	0.45	156.7 s	\$0.24	0.53	335.0 s	X	X	X	\$0.37	0.69	31.0 s	X	X	X	\$0.17	0.67	18.3 s
Q5: C	\$0.04	0.99	7.4 s	\$0.07	0.98	6.6 s	X	X	X	\$0.17	0.98	25.7 s	\$4·10 ⁻⁶	0.96	0.7 s	\$0.01	0.98	5.8 s
Q6: C	\$0.12	0.89	114.8 s	\$0.43	0.89	143.7 s	X	X	X	\$0.35	0.88	34.9 s	\$4·10 ⁻⁶	0.47	0.7 s	\$0.16	0.95	19.5 s
Q7: J	\$1.33	0.75	199.4 s	\$1.79	0.92	287.6 s	\$0.08	0.51	97.7 s	\$0.86	0.83	45.4 s	\$1·10 ⁻⁹	0.78	2·10 ⁻³ s	\$0.09	0.81	7.4 s
Q8: J	\$4·10 ⁻³	1.00	4.1 s	\$0.01	1.00	3.9 s	\$0.30	0.00	713.0 s	\$18.23	0.29	126.2 s	\$1·10 ⁻⁹	0.25	1·10 ⁻³ s	\$0.10	1.00	3.4 s
Q9: J	\$0.06	0.55	243.4 s	\$0.10	0.49	44.9 s	\$0.31	0.00	872.2 s	\$0.21	0.58	48.6 s	\$1·10 ⁻⁹	0.41	1·10 ⁻³ s	\$0.07	0.83	18.8 s
Q10: F J	\$0.21	0.00	519.5 s	\$1.02	0.06	1192.5 s	X	X	X	X	X	X	\$1·10 ⁻⁵	X	0.7 s	\$0.65	0.34	90.1 s
Q11: F J	\$0.27	0.78	158.7 s	\$0.61	0.73	132.4 s	X	X	X	X	X	X	X	X	X	\$1.68	0.40	131.3 s
Q12: F M	\$0.10	0.60	36.4 s	\$0.14	0.00	31.9 s	X	X	X	\$0.10	0.97	31.1 s	\$6·10 ⁻⁶	0.85	0.7 s	\$0.08	0.95	32.1 s
Q13: F	\$0.24	0.74	274.8 s	\$0.44	0.74	238.3 s	X	X	X	\$0.38	0.70	22.4 s	\$2·10 ⁻⁵	0.58	1.0 s	\$0.13	0.72	11.6 s
Q14: F J R	\$0.23	0.87	178.2 s	X	X	X	X	X	X	\$4.26	0.37	73.6 s	\$5·10 ⁻⁶	0.00	0.7 s	\$4.83	1.00	87.3 s
Avg	\$0.22	0.74	149.2 s	\$0.42	0.70	192.5 s	\$0.21	0.43	363.7 s	\$2.42	0.67	44.8 s	\$3·10 ⁻³	0.57	10.8 s	\$0.54	0.80	38.2 s
(c) Wildlife Scenario										DASE Preprocess			\$0.01		53.9 s	\$0.01		53.9 s
Q1: F	\$0.11	0.79	92.4 s	\$0.13	0.79	32.8 s	\$0.11	0.79	19.6 s	\$0.11	0.79	32.0 s	\$1·10 ⁻⁹	0.79	5·10 ⁻⁴ s	\$0.02	0.78	10.1 s
Q2: F	X	X	X	\$0.01	0.17	2.8 s	\$0.01	0.14	4.5 s	\$0.01	0.19	9.4 s	\$1·10 ⁻⁹	0.83	5·10 ⁻⁴ s	\$3·10 ⁻³	0.50	6.2 s
Q3: F L	\$0.11	1.00	99.4 s	\$0.13	0.00	22.7 s	\$0.03	0.00	4.3 s	\$0.11	0.00	25.7 s	\$1·10 ⁻⁹	0.00	8·10 ⁻⁴ s	\$0.02	1.00	9.0 s
Q4: F L	X	X	X	\$0.01	0.00	2.7 s	\$1·10 ⁻³	0.00	1.3 s	\$0.01	1.00	9.4 s	\$1·10 ⁻⁹	1.00	8·10 ⁻⁴ s	\$3·10 ⁻³	1.00	6.1 s
Q5: F	X	X	X	\$0.13	0.75	13.5 s	\$0.01	0.75	2.3 s	\$0.12	0.75	19.2 s	\$1·10 ⁻⁹	1.00	1·10 ⁻³ s	\$0.02	0.85	7.0 s
Q6: F	X	X	X	\$0.13	0.00	19.3 s	\$0.06	0.50	13.2 s	\$0.12	0.20	24.3 s	\$1·10 ⁻⁹	0.00	1·10 ⁻³ s	\$0.06	0.40	13.9 s
Q7: F	\$0.23	1.00	188.3 s	\$0.13	1.00	43.9 s	\$0.20	1.00	28.8 s	\$0.22	1.00	24.6 s	\$1·10 ⁻⁹	0.00	1·10 ⁻³ s	\$0.14	1.00	19.9 s
Q8: F	X	X	X	\$0.13	0.75	34.9 s	\$0.12	0.75	23.8 s	\$0.23	0.75	35.5 s	\$1·10 ⁻⁹	0.00	2·10 ⁻³ s	\$0.16	0.74	24.8 s
Q9: F	X	X	X	\$0.13	0.57	19.1 s	\$0.08	0.67	17.2 s	\$0.12	0.59	37.6 s	\$1·10 ⁻⁹	0.00	1·10 ⁻³ s	\$0.06	0.40	19.5 s
Q10: F L	\$0.11	1.00	87.8 s	\$0.13	0.00	19.6 s	\$0.03	0.00	4.4 s	\$0.11	0.00	40.3 s	\$1·10 ⁻⁹	1.00	8·10 ⁻⁴ s	\$0.06	0.25	24.2 s
Avg	\$0.14	0.94	117.0 s	\$0.11	0.40	21.1 s	\$0.07	0.46	11.9 s	\$0.12	0.52	25.8 s	\$9·10 ⁻⁴	0.46	4.9 s	\$0.05	0.69	17.7 s
(d) MMQA Scenario										DASE Preprocess			\$0.01		50.1 s	\$0.01		50.1 s
Q1: M	\$0.02	1.00	7.4 s	\$3·10 ⁻³	1.00	4.5 s	X	X	X	\$0.01	1.00	14.1 s	X	X	X	\$0.01	1.00	14.1 s
Q2a: J	\$0.89	0.83	169.6 s	\$1.04	1.00	135.9 s	X	X	X	\$0.08	0.00	53.8 s	\$1·10 ⁻⁵	0.08	1.0 s	\$0.04	0.57	28.6 s
Q2b: J	\$0.89	0.83	166.8 s	\$1.04	1.00	133.8 s	X	X	X	\$0.12	0.00	38.4 s	\$1·10 ⁻⁵	0.08	1.0 s	\$0.06	0.57	21.4 s
Q3a: F	\$0.01	0.83	4.2 s	\$0.02	0.80	4.5 s	\$0.01	0.75	11.0 s	\$0.01	0.72	19.6 s	\$5·10 ⁻⁶	0.38	0.7 s	\$9·10 ⁻³	0.72	20.5 s
Q3f: F	\$0.01	1.00	4.2 s	\$0.02	1.00	8.0 s	\$0.01	1.00	7.0 s	\$0.01	0.67	22.3 s	\$5·10 ⁻⁶	0.09	0.7 s	\$0.01	0.67	22.3 s
Q4: M	X	X	X	\$5·10 ⁻³	0.54	1.2 s	X	X	X	\$2·10 ⁻³	0.60	9.7 s	X	X	X	\$2·10 ⁻³	0.61	9.5 s
Q5: M	\$1·10 ⁻³	1.00	0.48 s	\$1·10 ⁻³	1.00	0.50 s	X	X	X	X	X	X	X	X	X	X	X	X
Q6a: F	\$0.01	1.00	4.4 s	\$0.02	1.00	4.0 s	\$2·10 ⁻³	0.33	5.6 s	\$4·10 ⁻³	0.03	18.9 s	\$5·10 ⁻⁶	0.04	0.7 s	\$3·10 ⁻³	1.00	15.8 s
Q6b: F	\$0.01	1.00	3.8 s	\$0.02	1.00	4.1 s	\$2·10 ⁻³	1.00	5.5 s	\$4·10 ⁻³	0.04	17.3 s	\$5·10 ⁻⁶	0.01	0.7 s	\$4·10 ⁻³	0.87	20.4 s
Q6c: F	\$0.01	1.00	4.6 s	\$0.02	1.00	4.7 s	\$2·10 ⁻³	0.53	13.4 s	\$4·10 ⁻³	0.13	17.4 s	\$5·10 ⁻⁶	0.10	0.7 s	\$4·10 ⁻³	0.66	19.1 s
Q7: J	\$13.61	0.32	2311.4 s	\$15.65	0.31	2101.6 s	X	X	X	\$1.18	0.00	91.7 s	\$1·10 ⁻⁵	0.10	0.7 s	\$0.04	1.00	6.9 s
Avg	\$1.55	0.88	267.7 s	\$1.62	0.87	218.4 s	\$5·10 ⁻³	0.72	8.5 s	\$0.14	0.31	30.3 s	\$1·10 ⁻³	0.11	6.3 s	\$0.02	0.76	20.8 s
(e) Cars Scenario										DASE Preprocess			\$2.50		1300 s	\$2.50		1300 s
Q1: F	\$1.74	0.90	550.0 s	\$2.44	0.69	465.6 s	\$1.37	0.81	829.7 s	\$1.44	0.71	61.7 s	\$5·10 ⁻⁶	0.66	0.9 s	\$0.31	0.78	22.0 s
Q2: F	X	X	X	\$4·10 ⁻³	0.00	4.1 s	\$0.01	0.09	14.1 s	\$0.01	0.08	14.1 s	\$5·10 ⁻⁶	0.00	0.7 s	\$1·10 ⁻³	0.16	5.1 s
Q3: F L	\$0.60	0.90	456.2 s	\$0.01	0.92	6.1 s	X	X	X	\$1.66	1.00	36.4 s	\$5·10 ⁻⁶	1.00	0.7 s	\$2·10 ⁻³	1.00	4.3 s
Q4: F	\$1.71	0.99	822.0 s	\$2.41	0.99	443.6 s	\$1.34	1.00	768.8 s	\$1.41	0.99	68.7 s	\$5·10 ⁻⁶	0.99	0.9 s	\$0.31	0.99	23.4 s
Q5: F	X	X	X	\$0.01	1.00	6.3 s	\$1.61	1.00	483.7 s	\$1.47	1.00	58.9 s	\$1·10 ⁻⁵	0.50	1.1 s	\$7·10 ⁻³	1.00	5.1 s
Q6: F J	X	X	X	\$2.51	0.96	427.3 s	\$1.96	0.97	775.4 s	\$2.00	0.96	44.3 s	\$1·10 ⁻⁵	0.88	1.3 s	\$0.89	0.96	41.2 s
Q7: F	X	X	X	\$4.47</														

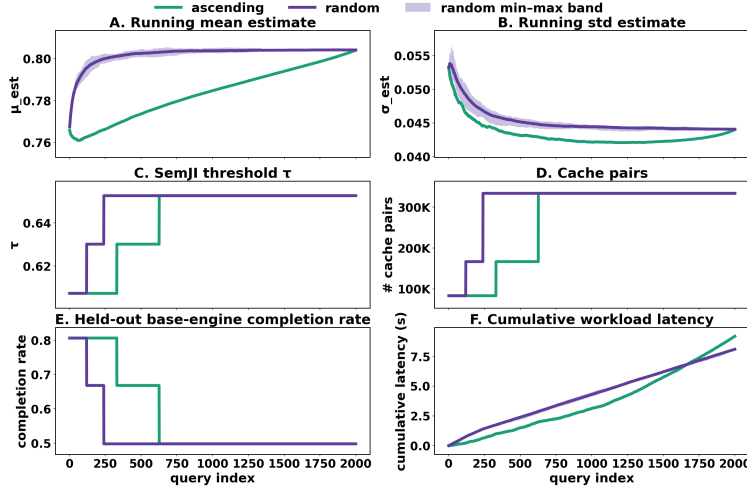


Figure 7: SEMJI evolution test.

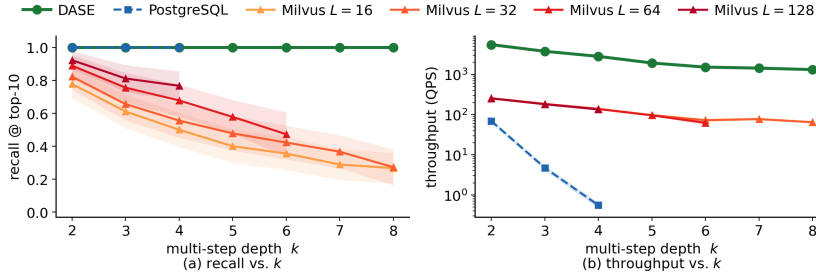


Figure 8: Number of Join Steps vs recall in NFCorpus

It retains the top- n partial paths per directed-edge state (remembering the preceding vertex to exclude immediate backtracking); its worst-case time is $O(kn \sum_v \deg(v)^2)$, or $O(kn|E|)$ for bounded-degree graphs.

Both baselines fail as k grows (Figure 8). PostgreSQL is exact but its k -way join explodes, collapsing from 70 to 0.55 QPS over $k=2 \dots 4$ and timing out for $k \geq 5$. Milvus’s qualifying sets are rarely nearest neighbors, so its recall decays ($L=16$: $0.78 \rightarrow 0.27$) and its L^k enumeration turns the accurate settings infeasible ($L=128$ by $k=6$). DASE instead keeps full recall for every k while sustaining 5,540 down to 1,321 QPS — one-to-four orders of magnitude above the planner and never infeasible. The gap *widens* with k : the baselines time out or blow up while DASE stays flat, confirming that join- and predicate-aware materialization pays off exactly in the multi-step regime.

5.5 Filter Sets in the ANN Index

Table 5 evaluates predicate-aware HNSW search strategies. The exhaustive baseline returns the exact top- k answer (recall 1.0), but at low throughput (0.23 QPS). To accelerate filtering, we compare Bitmap and Bloom filter structures, finding that the Bitmap approach consistently delivers better recall and higher QPS across all configurations. Within the Bitmap approach, we further evaluate graph traversal methods. The 2-hop adaptive strategy yields the optimal trade-off: by dynamically exploring extended neighbors only when valid nodes are sparse, it achieves a roughly 40× speedup

Table 4: SEMJI Construction.

Method	Recall	Time
Exact Construction	1.00	~81.2 h
Iterative	0.98	14.0 m
Batched	0.98	3.0 m
Fully Vectorized	0.98	1.6 m

Table 5: Filtered search results.

Method	Setting	Recall	QPS
baseline	-	1.0	0.23
Bitmap	2-hop on	0.905	9.6
	2-hop off	0.884	10.5
	adaptive	0.903	10.1
Bloom	4 bits/element	0.817	9.1
	12 b/element	0.872	8.8
	20 b/element	0.883	8.2

Table 6: Component ablation on IMDB. Each mechanism is disabled in isolation on the workload that exercises it; we report recall / throughput.

Mechanism	Variant	Recall	QPS
SEMJI (W5)	DASE (with SEMJI)	0.95	92.0
	– SEMJI (on-the-fly join)	0.71	5.3
Stream selection (W3)	DASE (adaptive)	0.98	57.8
	– adaptive (round-robin)	0.98	29.8

(10.1 QPS) over the baseline while maintaining a highly competitive recall of 0.903. This adaptive mechanism balances the speed of the “2-hop off” method with the accuracy of the “always-on” method.

5.6 DASE Ablation Study

We isolate DASE’s two central algorithmic contributions by disabling one at a time on the workload that exercises it in isolation, holding everything else fixed (Table 6; IMDB, 100 queries, warm cache, same host).

SEMJI join materialization (Table 6, top). On the pure semantic-join workload W5 we keep DASE’s seed streamer unchanged and switch *only* the partner check: the full engine probes the materialized SEMJI index, while the ablated variant resolves each candidate’s partner on the fly with an ANN query against the partner table. Removing SEMJI is strictly worse on *both* axes — recall drops from 0.95 to 0.71 and throughput falls 17× ($92.0 \rightarrow 5.3$ QPS) — because without the precomputed join the partner test must either approximate (missing qualifying partners, hence the recall loss) or run exact (far slower still). Materializing the join is what makes the join-bearing workloads both accurate and fast.

Distribution-aware stream selection (Table 6, bottom). On the pure multi-scoring workload W3 we replace DASE’s threshold-algorithm stream selector — which extends the stream with the largest expected gap reduction $G_j = w_j / \text{PDF}(x_j)$ — with vanilla round-robin extension. At the exact operating point ($\epsilon=0$, recall 0.98) adaptive selection delivers 1.9× the throughput (57.8 vs 29.8 QPS):

choosing the right stream closes the termination gap in far fewer extension rounds. The gain is largest exactly here, in the high-recall regime where many rounds are needed, and concentrates on queries whose signals have heterogeneous score distributions.

5.7 Robustness to Correlated Embeddings

The *materialization radius* τ_{cap} retains every endpoint pair within $\text{dist}(a, b) \leq \tau_{cap}$; SEMJI is the union of these per-endpoint neighborhoods. However, in a multi-predicate query it is not one ball around the results. A sample-derived cutoff can select the close-pair tail, but anisotropy can make a raw-distance cutoff unreliable. We therefore investigated the distribution of distances for the IMDB and MOLECULE W7/W8 join endpoints. Their covariance participation ratios are only 75–93 and 112–166 of 1,536 dimensions (i.e., 5–11% effective variance-carrying directions).

We then consider different methods for handling this anisotropy. For each corpus, we ran 200 queries over two independent 3K-by-3K reservoir samples (400 query-sample units per arm). During execution, we apply predicates before ranking and macro-average $\text{recall@20}/\text{precision@20}$ against each arm’s exhaustive predicate-filtered top-20 oracle; these are candidate-retrieval, not final-answer, measures. We use a 95% two-stage percentile bootstrap over endpoint samples and then queries. The raw-workload Recall/Precision intervals are 0.764–0.825/0.815–0.868 (IMDB) and 0.459–0.554/0.648–0.749 (MOLECULE); ABTT’s are 0.982–0.998/0.999–1.000 and 0.954–0.981/0.954–0.981. Each arm calibrates τ_{cap} in its own retrieval space; PCA uses 0.1 shrinkage and ABTT removes one leading component, both fit on each sample’s 6K endpoints.

Table 7: Real predicate-filtered SEMJI quality. Each R/P entry is recall@20 / precision@20 relative to that arm’s exhaustive oracle. The pairs column gives the materialized fraction for IMDB / MOLECULE; it is not a ratio.

Arm	IMDB Rec./Prec.	MOLECULE Rec./Prec.	Pairs % (I/M)
Raw workload cutoff	0.795 / 0.842	0.506 / 0.700	0.60% / 0.03%
Raw $\mu - 3\sigma$	0.920 / 0.929	0.867 / 0.867	0.42% / 0.62%
PCA whitening	0.910 / 0.920	0.868 / 0.868	0.44% / 0.47%
ABTT	0.991 / 1.000	0.969 / 0.969	1.09% / 0.93%
Raw radius control	0.954 / 0.962	0.881 / 0.881	1.09% / 0.93%

We observe from Table 7 that the raw workload cutoff strategy loses substantial coverage, especially on MOLECULE. **ABTT** has the highest within-arm coverage (0.991/1.000 on IMDB and 0.969/0.969 on MOLECULE). At the same materialized-pair fraction, the **raw radius control** remains 3.7 and 8.8 recall points lower, respectively; transformed-space calibration therefore improves which pairs are retained, not merely their number.

6 Related Work

Semantic-operator systems. LOTUS [30] and Palimpsest [26] develop declarative operations whose predicates, joins, or transformations may require LLM-based evaluation. ThalamusDB [20, 39] similarly applies approximate query processing to multimodal data. SemBench [23] compares these systems and others on semantic-filter, merge, and join tasks. DASE is complementary to these systems: it assumes that a semantic-query planner has supplied a retrieval plan and scoring function, then constructs and ranks a high-recall candidate set before final semantic judgment. Our DASE + BigQuery configuration instantiates this prefiltering role.

Record linking. Increasingly, record linking [10] tasks are being addressed using embeddings and LLMs [51]. SEMJI can be viewed as a materialized snapshot of links above a threshold.

Hybrid DBMSs. Vector DBMSs such as Milvus [42] combine vector retrieval with scalar filtering, while filtered-ANN work such as ACORN [31] and Filtered-DiskANN [16] improve **point-query vector search** under structured predicates. These methods are useful building blocks for DASE’s filtered score streams, but their central access pattern is one query vector against one indexed collection. Our targets include M-to-N embedding-similarity joins between distinct relational rows, and exact joins that connect those rows into a pattern, with a monotone score that combines several lookup and join affinities. SEMJI materializes a sparse subset of those cross-table pairs so that the resulting pattern can be evaluated with relational operations rather than repeated point lookups.

Ranked join processing. Threshold-style top- k algorithms [12, 21, 40] combine sorted score streams with random access and stopping bounds. DASE adopts this execution principle: an embedding-similarity-join score belongs to a pair of rows that must be connected through the query pattern before it contributes to an output tuple. The materialized join relation, filtered score streams, and Multi-Scoring Aggregator provide the access paths and bound-aware coordination needed for this setting.

Embedding geometry. Embedding whitening and anisotropy correction are well studied for NLP sentence and word representations [2, 11, 28, 38]. We use these techniques in a different role: when anisotropic or shared-context endpoint embeddings make a distribution-derived materialization radius unreliable, DASE separately calibrates a transformed embedding space and constructs the corresponding embedding-similarity join index there.

7 Conclusions

Multi-step reasoning queries frequently arise in agentic data engineering tasks such as scientific discovery. We introduced SEMJI, which facilitates efficient candidate exploration in top- k multi-step reasoning queries, along with ANN extensions to enable batched requests. We developed DASE, a query processing strategy that jointly optimizes attribute predicates and vector similarity scoring across joins. We conducted an extensive evaluation on existing semantic benchmarks and a scientific-discovery workload that captures the structure of multi-step reasoning queries. Our results demonstrate the retrieval benefits and approximation trade-offs of SEMJI. We also evaluated a robustness treatment for anisotropic endpoint embeddings, where a raw distribution-derived SEMJI materialization radius can miss useful links.

We hope that our formalization, methods, and benchmarks serve as a foundation for continued progress on query systems that bridge structured and unstructured data at scale. In ongoing work, we are focused on developing a high-level planner that uses costs to break agentic plans into stages with multi-step reasoning components.

References

- [1] Eric Anderson, Jonathan Fritz, Austin Lee, Bohou Li, Mark Lindblad, Henry Lindeman, Alex Meyer, Parth Parmar, Tanvi Ranade, Mehul A. Shah, Ben Sowell, Dan G. Tecuci, Vinayak Thapliyal, and Matt Welsh. 2024. The Design of an LLM-powered Unstructured Analytics System. *ArXiv abs/2409.00847* (2024). <https://api.semanticscholar.org/CorpusID:272368251>

- [2] Sanjeev Arora, Yingyu Liang, and Tengyu Ma. 2017. A simple but tough-to-beat baseline for sentence embeddings. In *International conference on learning representations*.
- [3] Burton H. Bloom. 1970. Space/Time Trade-offs in Hash Coding with Allowable Errors. *CACM* 13, 7 (July 1970), 422–426.
- [4] Vera Boteva, Demian Gholipour, Artem Sokolov, and Stefan Riezler. 2016. A full-text learning to rank dataset for medical information retrieval. In *European Conference on Information Retrieval*. Springer, 716–722.
- [5] Huiping Cao, Yan Qi, K Selçuk Candan, and Maria Luisa Sapino. 2010. Feedback-driven result ranking and query refinement for exploring semi-structured data collections. In *EDBT*. 3–14.
- [6] Mingyu Chen, Guibin Zhang, Heng Chang, Yuchen Guo, and Shiji Zhou. 2026. A-MapReduce: Executing Wide Search via Agentic MapReduce. *arXiv preprint arXiv:2602.01331* (2026).
- [7] Xu Chen, Zheng Qin, Yongfeng Zhang, and Tao Xu. 2016. Learning to rank features for recommendation over multiple categories. In *Proceedings of the 39th International ACM SIGIR conference on Research and Development in Information Retrieval*. 305–314.
- [8] Seyone Chithrananda, Gabriel Grand, and Bharath Ramsundar. 2020. ChemBERTa: Large-Scale Self-Supervised Pretraining for Molecular Property Prediction. doi:10.48550/arXiv.2010.09885 arXiv:2010.09885.
- [9] Peter Christen. 2011. A survey of indexing techniques for scalable record linkage and deduplication. *IEEE transactions on knowledge and data engineering* 24, 9 (2011), 1537–1555.
- [10] Peter Christen. 2012. *Data Matching: Concepts and Techniques for Record Linkage, Entity Resolution, and Duplicate Detection*. Springer.
- [11] Kavin Ethayarajh. 2019. How Contextual are Contextualized Word Representations? Comparing the Geometry of BERT, ELMo, and GPT-2 Embeddings. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. 55–65.
- [12] Ronald Fagin, Amnon Lotem, and Moni Naor. 2003. Optimal aggregation algorithms for middleware. *Journal of computer and system sciences* 66, 4 (2003), 614–656.
- [13] Sérgio Fernandes and Jorge Bernardino. 2015. What is bigquery?. In *Proceedings of the 19th International Database Engineering & Applications Symposium*. 202–203.
- [14] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Qianyu Guo, Meng Wang, and Haofen Wang. 2023. Retrieval-Augmented Generation for Large Language Models: A Survey. *ArXiv abs/2312.10997* (2023). <https://api.semanticscholar.org/CorpusID:266359151>
- [15] Ali Essam Ghareeb, Benjamin Chang, Ludovico Mitchener, Angela Yiu, Caralyn J Szostkiewicz, Dmytro Shved, Gavin J Gyimesi, Jon M Laurent, Samantha M Wright, Muhammed T Razzak, et al. 2026. A multi-agent system for automating scientific discovery. *Nature* (2026), 1–3.
- [16] Siddharth Gollapudi, Neel Karia, Varun Sivashankar, Ravishankar Krishnaswamy, Nikit Begwani, Swapnil Raz, Yiyong Lin, Yin Zhang, Neelam Mahapatro, Premkumar Srinivasan, and others. 2023. Filtered-diskann: Graph algorithms for approximate nearest neighbor search with filters. In *Proceedings of the ACM Web Conference 2023*. 3406–3416.
- [17] Juraj Gottweis, Wei-Hung Weng, Alexander Daryin, Tao Tu, Petar Sirkovic, Artiom Myaskovsky, Grzegorz Glowaty, Felix Weissenberger, Alessio Orlando, Dan Popovici, et al. 2026. Accelerating scientific discovery with Co-Scientist. *Nature* (2026), 1–3.
- [18] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shiron Ma, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Hanwei Xu, Honghui Ding, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jingchang Chen, Jingyang Yuan, Jinhao Tu, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaichao You, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Lian Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingxu Zhou, Meng Li, Miaojuan Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qishi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shutong Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Li, Yongqiang Guo, Yuan Ou, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. 2025. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature* 645, 8081 (Sept. 2025), 633–638. doi:10.1038/s41586-025-09422-z
- [19] Ziyang Huang, Haolin Ren, Xiaowei Yuan, Jiawei Wang, Zhongtao Jiang, Kun Xu, Shizhu He, Jun Zhao, and Kang Liu. 2026. WideSeek: Advancing Wide Research via Multi-Agent Scaling. *arXiv preprint arXiv:2602.02636* (2026).
- [20] Saehan Jo and Immanuel Trummer. 2024. ThalamusDB: Approximate Query Processing on Multi-Modal Data. *Proceedings of the ACM on Management of Data* 2, 3 (May 2024), 1–26. doi:10.1145/3654989
- [21] Benny Kimmelfeld and Yehoshua Sagiv. 2006. Finding and approximating top-k answers in keyword proximity search. In *PODS*. 173–182.
- [22] Tian Lan, Bin Zhu, Qianhuai Jia, Junyang Ren, Haijun Li, Longyue Wang, Zhao Xu, Weihua Luo, and Kaifu Zhang. 2025. Deepwidesearch: Benchmarking depth and width in agentic information seeking. *arXiv preprint arXiv:2510.20168* (2025).
- [23] Jiale Lao, Andreas Zimmerer, Olga Ovcharenko, Tianji Cong, Matthew Russo, Gerardo Vitagliano, Michael Cochez, Fatma Özcan, Gautam Gupta, Thibaud Hottelier, H. V. Jagadish, Kris Kissel, Sebastian Schelter, Andreas Kipf, and Immanuel Trummer. 2025. SemBench: A Benchmark for Semantic Query Processing Engines. doi:10.48550/arXiv.2511.01716 arXiv:2511.01716.
- [24] Chengkai Li, Kevin Chen-Chuan Chang, Ihab F. Ilyas, and Sumin Song. 2005. RankSQL: Query Algebra and Optimization for Relational Top-k Queries. In *SIGMOD*. 131–142.
- [25] Hang Li. 2011. *Learning to Rank for Information Retrieval and Natural Language Processing*. Morgan Claypool.
- [26] Chunwei Liu, Matthew Russo, Michael Cafarella, Lei Cao, Peter Baile Chen, Zui Chen, Michael Franklin, Tim Kraska, Samuel Madden, Rana Shahout, et al. 2025. Palimpsest: Optimizing ai-powered analytics with declarative query processing. In *Proceedings of the Conference on Innovative Database Research (CIDR)*. 2.
- [27] Duo Lu, Helena Caminal, Manos Chatzakos, Yannis Papakonstantinou, Yannis Chronis, Vaibhav Jain, and Fatma Özcan. 2026. An in-depth study of filter-agnostic vector search on a postgresql database system: [experiments & analysis]. *Proceedings of the ACM on Management of Data* 4, 3 (SIGMOD (2026)), 1–26.
- [28] Jiaqi Mu and Pramod Viswanath. 2018. All-but-the-Top: Simple and Effective Postprocessing for Word Representations. In *International Conference on Learning Representations (ICLR)*.
- [29] Rodrigo Nogueira and Kyunghyun Cho. 2020. Passage Re-ranking with BERT. doi:10.48550/arXiv.1901.04085 arXiv:1901.04085.
- [30] Liana Patel, Siddharth Jha, Melissa Pan, Harshit Gupta, Parth Asawa, Carlos Guestrin, and Matei Zaharia. 2025. Semantic Operators and Their Optimization: Enabling LLM-Based Data Processing with Accuracy Guarantees in LOTUS. *Proceedings of the VLDB Endowment* 18, 11 (2025), 4171–4184.
- [31] Liana Patel, Peter Kraft, Carlos Guestrin, and Matei Zaharia. 2024. Acorn: Performant and predicate-agnostic search over vector embeddings and structured data. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–27.
- [32] Bocci Peng, Yun Zhu, Yongchao Liu, Xiaoho Bo, Haizhou Shi, Chuntao Hong, Yan Zhang, and Siliang Tang. 2024. Graph Retrieval-Augmented Generation: A Survey. doi:10.48550/arXiv.2408.08921 arXiv:2408.08921.
- [33] Pinecone Systems. 2026. Pinecone: The Vector Database for AI Applications. <https://www.pinecone.io/>. Accessed: 2026-04-15.
- [34] David Rogers and Mathew Hahn. 2010. Extended-Connectivity Fingerprints. *Journal of Chemical Information and Modeling* 50, 5 (May 2010), 742–754. doi:10.1021/ci100050t
- [35] Matthew Russo, Sivaprasad Sudhir, Gerardo Vitagliano, Chunwei Liu, Tim Kraska, Samuel Madden, and Michael Cafarella. 2025. Abacus: A Cost-Based Optimizer for Semantic Operator Systems. *arXiv preprint arXiv:2505.14661* (2025).
- [36] Ahmmad OM Saleh, Gokhan Tur, and Yucel Saygin. 2024. SG-RAG: Multi-hop question answering with large language models through knowledge graphs. In *Proceedings of the 7th International Conference on Natural Language and Speech Processing (ICNLSP 2024)*. 439–448.
- [37] Shreya Shankar, Tristan Chambers, Tarak Shah, Aditya G Parameswaran, and Eugene Wu. 2025. DocETL: Agentic Query Rewriting and Evaluation for Complex Document Processing. *Proceedings of the VLDB Endowment* 18, 9 (2025), 3035–3048.
- [38] Jianlin Su, Jiarun Cao, Weijie Liu, and Yangyiwen Ou. 2021. Whitening Sentence Representations for Better Semantics and Faster Retrieval. *arXiv preprint arXiv:2103.15316* (2021). doi:10.48550/arXiv.2103.15316 arXiv:2103.15316.
- [39] Immanuel Trummer. 2025. Implementing Semantic Join Operators Efficiently. doi:10.48550/arXiv.2510.08489 arXiv:2510.08489.
- [40] Nikolaos Tziavelis, Wolfgang Gatterbauer, and Mirek Riedewald. 2024. Ranked Enumeration for Database Queries. *ACM SIGMOD Record* 53, 3 (Nov. 2024), 6–19. doi:10.1145/3703922.3703924
- [41] Roman Vershynin. 2018. *High-dimensional probability: An introduction with applications in data science*. Vol. 47. Cambridge university press.

- [42] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xianguo Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, Kun Yu, Yuxing Yuan, Yinghao Zou, Jiquan Long, Yudong Cai, Zhenxiang Li, Zhifeng Zhang, Yihua Mo, Jun Gu, Ruiyi Jiang, Yi Wei, and Charles Xie. 2021. Milvus: A Purpose-Built Vector Data Management System. In *Proceedings of the 2021 International Conference on Management of Data*. ACM, Virtual Event China, 2614–2627. doi:10.1145/3448016.3457550
- [43] Weaviate. 2026. Weaviate: An Open-Source Cloud-Native Vector Database. <https://github.com/weaviate/weaviate>. Accessed: 2026-04-15.
- [44] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, and others. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
- [45] David Weininger. 1988. SMILES, a chemical language and information system. 1. Introduction to methodology and encoding rules. *Journal of chemical information and computer sciences* 28, 1 (1988), 31–36.
- [46] Ryan Wong, Jiawei Wang, Junjie Zhao, Li Chen, Yan Gao, Long Zhang, Xuan Zhou, Zuo Wang, Kai Xiang, Ge Zhang, Wenhao Huang, Yang Wang, and Ke Wang. 2025. WideSearch: Benchmarking Agentic Broad Info-Seeking. arXiv:2508.07999 [cs.CL] <https://arxiv.org/abs/2508.07999>
- [47] Zelai Xu, Zhexuan Xu, Ruize Zhang, Chunyang Zhu, Shi Yu, Weilin Liu, Quanlu Zhang, Wenbo Ding, Chao Yu, and Yu Wang. 2026. WideSeek-R1: Exploring Width Scaling for Broad Information Seeking via Multi-Agent Reinforcement Learning. arXiv:2602.04634 [cs.AI] <https://arxiv.org/abs/2602.04634>
- [48] Zhepeng Yan, Nan Zheng, Zachary Ives, Partha Talukdar, and Cong Yu. 2013. Actively Soliciting Feedback for Query Answers in Keyword Search-Based Data Integration. *PVLDB* (2013).
- [49] Rui Yang, Lin Song, Yanwei Li, Sijie Zhao, Yixiao Ge, Xiu Li, and Ying Shan. 2023. Gpt4tools: Teaching large language model to use tools via self-instruction. *Advances in Neural Information Processing Systems* 36 (2023), 71995–72007.
- [50] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022. ReAct: Synergizing Reasoning and Acting in Language Models. *International Conference on Learning Representations* abs/2210.03629 (2022).
- [51] Alexandros Zeakis, George Papadakis, Dimitrios Skoutas, and Manolis Koubarakis. 2025. Avenger: Ensembling and Fine-Tuning LLMs for SELECT Prompts in Entity Resolution. In *European Semantic Web Conference*. Springer, 301–320.